November 6, 2026 at 22:19

1. Introduction. Unit propagation is the workhorse of every `SAT` solver, and it is cheap. Exercises 442–444 of Section 7.2.2.2 ask what happens when we make it progressively more expensive, and they sort all families of clauses into a hierarchy according to how much propagation they need before they give up their secrets.

Exercise 442 defines the relation $\vdash_k$. Let F be a family of clauses and l a literal. If $(l_1, l_2, \ldots, l_p)$ is a sequence of literals, write $L_q^- = \{l_1, \ldots, l_{q-1}, \bar{l}_q\}$ for $1 \le q \le p$. Then

$$\begin{aligned} F \vdash_0 l &\iff \epsilon \in F; \\ F \vdash_{k+1} l &\iff F \mid L_1^- \vdash_k \epsilon,\ \ldots,\ \text{and } F \mid L_p^- \vdash_k \epsilon \\ &\qquad \text{for some strictly distinct literals } l_1, \ldots, l_p \text{ with } l_p = l; \\ F \vdash_k \epsilon &\iff F \vdash_k l \text{ and } F \vdash_k \bar{l} \text{ for some literal } l. \end{aligned}$$

Two words in that definition carry all the weight. "Strictly distinct" means that no two of $l_1, \ldots, l_p$ involve the same variable; and the literals need not occur in F at all. Both facts have consequences that this program ran into head first, and they are discussed where they arise.

2. Exercise 443 uses $\vdash_k$ to define two classes. A family F belongs to UC_k if $F \mid L \vdash \epsilon$ implies $F \mid L \vdash_k \epsilon$ for every set L of strictly distinct literals—whenever a partial assignment makes the clauses unsatisfiable, kth order propagation notices. And F belongs to PC_k if $F \mid L \vdash l$ implies $F \mid L \vdash_k l$ whenever $L \cup l$ is strictly distinct. Part (a) asks for a proof that

$$PC_0 \subset UC_0 \subset PC_1 \subset UC_1 \subset PC_2 \subset UC_2 \subset \cdots,$$

with every inclusion strict.

Exercise 444 is about the *single lookahead unit resolution* algorithm SLUR, which answers 'sat', 'unsat', or 'maybe'. Part (c) claims that SLUR never answers 'maybe' if and only if $F \in UC_1$.

3. These exercises are about tiny objects—a family of four clauses on three variables is already interesting—so the honest way to check them is to compute $\vdash_k$ exactly as defined, over every partial assignment, and then to test each claim on every family that fits in a small universe. That is what this program does. Nothing here is a heuristic: *derive* and *contra* are the relation itself, obtained by searching for the sequences $(l_1, \ldots, l_p)$ that the definition calls for.

The `sat` package of the directory above, a Go rendering of Knuth's `SAT13`, plays the part of the semantic oracle. Whenever the program needs to know whether $F \mid L$ is satisfiable it can ask either an exhaustive loop over the 2^n total assignments or the solver, with L supplied as assumption literals; the *oracle* mode checks that the two never disagree, which puts the package's assumption machinery through a few hundred thousand calls.

I wrote this on 16 September 2026.

4. Every claim is a mode.

```
package main
import (
    "context"
    "flag"
    "fmt"
    "math/bits"
    "math/rand/v2"
    "os"
    "strconv"
    "strings"
    "github.com/sjnam/sat"
)
⟨ Types 9 ⟩
⟨ Global variables 6 ⟩
⟨ Functions 7 ⟩
func main() {
    ⟨ Read the command line 5 ⟩
    if on("prop") {
        ⟨ Check exercise 442 30 ⟩
    }
    if on("hier") {
        ⟨ Check exercise 443 47 ⟩
    }
    if on("slur") {
        ⟨ Check exercise 444 57 ⟩
    }
    if on("honest") {
        ⟨ Check the honesty test of page 289 72 ⟩
    }
    if on("oracle") {
        ⟨ Check the tables against the sat package 77 ⟩
    }
    ⟨ Say how it went 8 ⟩
}
```

5. The default settings run every mode in a few seconds. The sample sizes grow with `-reps`.

⟨ Read the command line 5 ⟩ ≡

```
flag.StringVar(&mode, "mode", "all",
    "prop, hier, slur, honest, oracle, or all")
flag.IntVar(&reps, "reps", 3000, "random families per number of variables")
flag.Uint64Var(&seed, "seed", 20260916, "seed for the random families")
flag.Parse()
```

This code is used in section 4.

6. ⟨ Global variables 6 ⟩ ≡

```
var (
    mode string
    reps int
    seed uint64
    failures int
)
```

This code is also defined in sections 29, 53, 60.

This code is used in section 4.

7. Each claim is announced, checked, and marked. A single `FAIL` anywhere makes the program exit with a nonzero status, so that `make` notices.

⟨ Functions 7 ⟩ ≡

```
func on(m string) bool { return mode ≡ m ∨ mode ≡ "all" }
func claim(ok bool, format string, args ... any) {
    mark := "ok␣␣"
    if ¬ok {
        mark = "FAIL"
        failures++
    }
    fmt.Printf("␣␣%s␣␣%s\n", mark, fmt.Sprintf(format, args...))
}
```

This code is also defined in sections 10, 11, 12, 13, 15, 21, 23, 24, 27, 28, 37, 43, 44, 45, 46, 52, 54, 56, 63, 65, 68, 69, 74, 75, 78, 82, 83.

This code is used in section 4.

8. ⟨ Say how it went 8 ⟩ ≡

```
if failures > 0 {
    fmt.Printf("\n%d␣claims␣failed.\n", failures)
    os.Exit(1)
}
fmt.Println("\nEvery␣claim␣checked␣out.")
```

This code is used in section 4.

9. Clauses, families, and partial assignments. Following Knuth, the literal for variable v is $2v$ when positive and $2v+1$ when negative, so that complementation is $l \oplus 1$. A clause is a bitmask over literals, which makes a clause of at most fifteen variables a **uint32**; a family is a list of clauses. Knuth's own shorthand for the clause $(x_1 \vee x_2 \vee \bar{x}_3)$ is '12$\bar{3}$', and *litName* prints $\bar{3}$ as `3'`.

⟨ Types 9 ⟩ ≡

```
type fam struct {
    n int
    cl []uint32
}
```

This code is also defined in sections 14, 22.

This code is used in section 4.

10. ⟨ Functions 7 ⟩ +≡

```
func lit(v int, positive bool) int {
    if positive {
        return 2 * v
    }
    return 2 * v + 1
}
func litName(l int) string {
    s := strconv.Itoa(l >> 1)
    if l & 1 == 1 {
        s += "'"
    }
    return s
}
```

11. A family prints as Knuth writes it, and *parseFam* reads that notation back, so that the examples quoted in the answers can be typed in exactly as they are printed in the book.

⟨ Functions 7 ⟩ +≡

```
func (f fam) String() string {
    parts := make([]string, len(f.cl))
    for i, c := range f.cl {
        parts[i] = litsOf(c, false)
    }
    return "{" + strings.Join(parts, ",") + "}"
}
func litsOf(m uint32, commas bool) string {
    var b strings.Builder
    for l := 2; l < 32; l++ {
        if m & (1 << l) ≠ 0 {
            if commas ∧ b.Len() > 0 {
                b.WriteByte(',')
            }
            b.WriteString(litName(l))
        }
    }
    if b.Len() ≡ 0 {
        return "eps"
    }
    return b.String()
}
```

12. ⟨ Functions 7 ⟩ +≡

```
func parseFam(n int, s string) fam {
    f := fam{n: n}
    for _, w := range strings.Fields(s) {
        var c uint32
        for i := 0; i < len(w); i++ {
            v := int(w[i] - '0')
            positive := true
            if i + 1 < len(w) ∧ w[i + 1] ≡ '\'' {
                positive, i = false, i + 1
            }
            c |= 1 << lit(v, positive)
        }
        f.cl = append(f.cl, c)
    }
    return f
}
```

13. There are exactly 3^n clauses on n variables that are not tautological: each variable is absent, positive, or negative. The empty clause is one of them.

```
⟨ Functions 7 ⟩ +≡
  func allClauses(n int) []uint32 {
    out := []uint32{0}
    for v := 1; v ≤ n; v++ {
      next := make([]uint32, 0, 3 * len(out))
      for _, c := range out {
        next = append(next, c, c | 1 ≪ lit(v, true), c | 1 ≪ lit(v, false))
      }
      out = next
    }
    return out
  }
```

14. A set L of strictly distinct literals is the same thing as a partial assignment, and there are 3^n of those too. The program numbers them in base three, digit $v-1$ telling whether variable v is unset, true, or false, and then tabulates everything it will ever want to know about $F \mid L$.

```
⟨ Types 9 ⟩ +≡
  type ctx struct {
    f fam
    n int
    nA int   // how many partial assignments, namely 3^n
    pow3 []int
    tmask []uint32   // the literals that the assignment makes true
    fmask []uint32   // the literals it makes false
    free []uint32   // the literals on its unassigned variables
    unsat []bool   // is F | L unsatisfiable?
    eps []bool   // does F | L contain the empty clause?
    nres []int   // how many clauses of F are not yet satisfied
    lev []lvl
    lits []sat.Lit
    stamp []int32
    now int32
    queue []int
  }
```

15.
```
⟨ Functions 7 ⟩ +≡
  func newCtx(f fam) *ctx {
    c := &ctx{f: f, n: f.n}
    ⟨ Set up the powers of three 16 ⟩
    ⟨ Find the total assignments that satisfy F 17 ⟩
    ⟨ Tabulate every partial assignment 18 ⟩
    return c
  }
  func (c *ctx) digit(a, v int) int { return (a / c.pow3[v - 1]) % 3 }
```

16. ⟨ Set up the powers of three 16 ⟩ ≡

```
c.pow3 = make([]int, c.n + 1)
c.pow3[0] = 1
for i := 1; i ≤ c.n; i++ {
    c.pow3[i] = 3 * c.pow3[i − 1]
}
c.nA = c.pow3[c.n]
```

This code is used in section 15.

17. Satisfiability is decided by brute force over the 2^n total assignments. For $n \le 6$ that is faster than any solver could be, and the *oracle* mode checks it against the package anyway.

⟨ Find the total assignments that satisfy F 17 ⟩ ≡

```
satT := make([]bool, 1 ≪ c.n)
for t := 0; t < 1 ≪ c.n; t++ {
    var tm uint32
    for v := 1; v ≤ c.n; v++ {
        tm |= 1 ≪ lit(v, t & (1 ≪ (v − 1)) ≠ 0)
    }
    satT[t] = true
    for _, cl := range f.cl {
        if cl & tm ≡ 0 {
            satT[t] = false
            break
        }
    }
}
```

This code is used in section 15.

18. A clause is gone from $F \mid L$ once one of its literals is true; otherwise what remains is the clause minus the literals that L falsifies, and if nothing remains we have the empty clause.

```
⟨ Tabulate every partial assignment 18 ⟩ ≡
    ⟨ Make room for the tables 19 ⟩
    for a := 0; a < c.nA; a++ {
        var tm, fm, fr uint32
        for v := 1; v ≤ c.n; v++ {
            switch c.digit(a, v) {
            case 0:
                fr |= 1 ≪ lit(v, true) | 1 ≪ lit(v, false)
            case 1:
                tm |= 1 ≪ lit(v, true)
                fm |= 1 ≪ lit(v, false)
            case 2:
                tm |= 1 ≪ lit(v, false)
                fm |= 1 ≪ lit(v, true)
            }
        }
        c.tmask[a], c.fmask[a], c.free[a] = tm, fm, fr
        for _, cl := range f.cl {
            if cl & tm ≠ 0 {
                continue
            }
            c.nres[a]++
            if cl ⊘ fm ≡ 0 {
                c.eps[a] = true
            }
        }
        ⟨ Decide whether F | L is satisfiable 20 ⟩
    }
```

This code is used in section 15.

19. ⟨ Make room for the tables 19 ⟩ ≡

```
c.tmask = make([]uint32, c.nA)
c.fmask = make([]uint32, c.nA)
c.free = make([]uint32, c.nA)
c.unsat = make([]bool, c.nA)
c.eps = make([]bool, c.nA)
c.nres = make([]int, c.nA)
c.stamp = make([]int32, c.nA)
```

This code is used in section 18.

20. ⟨ Decide whether $F \mid L$ is satisfiable 20 ⟩ ≡

```
c.unsat[a] = true
for t := 0; t < 1 ≪ c.n ∧ c.unsat[a]; t++ {
    if ¬satT[t] {
        continue
    }
    ok := true
    for v := 1; v ≤ c.n ∧ ok; v++ {
        d, b := c.digit(a, v), t & (1 ≪ (v − 1)) ≠ 0
        ok = ¬((d ≡ 1 ∧ ¬b) ∨ (d ≡ 2 ∧ b))
    }
    if ok {
        c.unsat[a] = false
    }
}
```

This code is used in section 18.

21. Two small services. *assign* extends a partial assignment so that the given literal becomes true, returning -1 if its variable is already set; and *implies* is the semantic relation $F \mid L \vdash l$, which by definition holds exactly when $F \mid L \mid \bar{l}$ is unsatisfiable.

⟨ Functions 7 ⟩ +≡

```
func (c *ctx) assign(a, l int) int {
    v := l ≫ 1
    if c.digit(a, v) ≠ 0 {
        return −1
    }
    if l & 1 ≡ 0 {
        return a + c.pow3[v − 1]
    }
    return a + 2 ∗ c.pow3[v − 1]
}
func (c *ctx) implies(a, l int) bool {
    b := c.assign(a, l ⊕ 1)
    return b ≥ 0 ∧ c.unsat[b]
}
```

22. The relation of exercise 442. For each level k the program stores, for every partial assignment L, the set $\{l \mid F \mid L \vdash_k l\}$ and the truth of $F \mid L \vdash_k \epsilon$. Level zero is immediate from the definition: if $\epsilon \in F \mid L$ then every literal follows, and otherwise none does.

⟨ Types 9 ⟩ +≡

```
type lvl struct {
    derive []uint32
    contra []bool
}
```

23. ⟨ Functions 7 ⟩ +≡

```
func (c *ctx) buildLevels(K int) {
    c.lev = make([]lvl, K + 1)
    l0 := lvl{derive: make([]uint32, c.nA), contra: make([]bool, c.nA)}
    for a := 0; a < c.nA; a++ {
        if c.eps[a] {
            l0.derive[a], l0.contra[a] = c.free[a], true
        }
    }
    c.lev[0] = l0
    for k := 0; k < K; k++ {
        c.lev[k + 1] = c.nextLevel(c.lev[k])
    }
}
```

24. Now for the sequences. A prefix $(l_1, \ldots, l_{q-1})$ is itself a partial assignment B, legal if every step so far was legal; the next literal l_q may be appended when $F \mid B \mid \bar{l}_q \vdash_k \epsilon$, and then l_q is a literal that $F \mid L$ derives, because a sequence may stop anywhere. So the literals of $\{l \mid F \mid L \vdash_{k+1} l\}$ are found by a breadth-first search over the assignments reachable from L.

⟨ Functions 7 ⟩ +≡

```
func (c *ctx) nextLevel(prev lvl) lvl {
    next := lvl{derive: make([]uint32, c.nA), contra: make([]bool, c.nA)}
    for a := 0; a < c.nA; a++ {
        ⟨ Collect the literals derivable from a at this level 25 ⟩
        next.derive[a] = got
        ⟨ Decide whether the empty clause follows 26 ⟩
    }
    return next
}
```

25. The search never revisits an assignment, and the strictly distinct rule falls out of the representation: a prefix is an assignment, so a variable it has already used cannot be used again.

⟨ Collect the literals derivable from a at this level 25 ⟩ ≡

```
c.now++
c.queue = append(c.queue[:0], a)
c.stamp[a] = c.now
var got uint32
for qi := 0; qi < len(c.queue); qi++ {
    b := c.queue[qi]
    for v := 1; v ≤ c.n; v++ {
        if c.digit(b, v) ≠ 0 {
            continue
        }
        for _, l := range [2]int { lit(v, true), lit(v, false) }{
            if ¬prev.contra[c.assign(b, l ⊕ 1)] {
                continue
            }
            got |= 1 ≪ l
            if nb := c.assign(b, l); c.stamp[nb] ≠ c.now {
                c.stamp[nb], c.queue = c.now, append(c.queue, nb)
            }
        }
    }
}
```

This code is used in section 24.

26. Here is the first place where reading carefully pays. We want some literal l with $F \mid L \vdash_{k+1} l$ and $F \mid L \vdash_{k+1} \bar{l}$, and the definition does not ask l to occur in $F \mid L$. Take a variable v that doesn't occur there at all. Then $F \mid L \mid v = F \mid L \mid \bar{v} = F \mid L$, so the one-element sequences give $F \mid L \vdash_{k+1} v$ and $F \mid L \vdash_{k+1} \bar{v}$ precisely when $F \mid L \vdash_k \epsilon$. Hence $\vdash_k \epsilon$ always implies $\vdash_{k+1} \epsilon$—which is half of part (c), and it must be built in here rather than proved later, on pain of getting the tables wrong. (It was wrong at first.)

⟨ Decide whether the empty clause follows 26 ⟩ ≡

```
next.contra[a] = prev.contra[a]
for v := 1; v ≤ c.n; v++ {
    if got & (1 ≪ lit(v, true)) ≠ 0 ∧ got & (1 ≪ lit(v, false)) ≠ 0 {
        next.contra[a] = true
    }
}
```

This code is used in section 24.

27. Unit propagation, and failing literals. Part (a) asks us to check that $\vdash_1$ is unit propagation. The proof in the answer is one line: $F \mid L_q^-$ contains ϵ if and only if $F \mid l_1 \mid \cdots \mid l_{q-1}$ contains ϵ or the unit clause (l_q). So *up* propagates units in whatever order it finds them, reporting the literals it forced and whether it ran into the empty clause.

⟨ Functions 7 ⟩ +≡

```
func (c *ctx) up(a int)(int, bool, uint32) {
    var got uint32
    for {
        if c.eps[a] {
            return a, true, got
        }
        found := -1
        for _, cl := range c.f.cl {
            steps++
            if cl & c.tmask[a] != 0 {
                continue
            }
            if r := cl &^ c.fmask[a]; bits.OnesCount32(r) == 1 {
                found = bits.TrailingZeros32(r)
                break
            }
        }
        if found < 0 {
            return a, false, got
        }
        got, forced = got | 1 << found, forced + 1
        a = c.assign(a, found)
    }
}
```

28. Part (b) says that $\vdash_2$ is unit propagation plus failed literal elimination: if $F \not\vdash_1 l$ and $F \mid \bar{l} \vdash_1 \epsilon$, then l may be forced and the search for further reductions resumed.

⟨ Functions 7 ⟩ +≡

```
func (c *ctx) fle(a int)(bool, uint32) {
    a, bad, got := c.up(a)
    if bad {
        return true, got
    }
    for progress := true; progress; {
        progress = false
        for l := 2; l < 2 * c.n + 2 ∧ ¬progress; l++ {
            if c.free[a] & (1 ≪ l) ≡ 0 {
                continue
            }
            if _, dead, _ := c.up(c.assign(a, l ⊕ 1)); ¬dead {
                continue
            }
            got |= 1 ≪ l
            var g uint32
            if a, bad, g = c.up(c.assign(a, l)); bad {
                return true, got | g
            }
            got, progress = got | g, true
        }
    }
    return false, got
}
```

29. ⟨ Global variables 6 ⟩ +≡

```
var (
    steps int   // clauses looked at, for the bound in part 442(f)
    forced int  // literals forced in a lookahead, for part 444(d)
)
```

30. Checking exercise 442. Parts (a) through (d) are statements about all families, so they are checked on thousands of random ones, at every partial assignment and every level.

⟨ Check exercise 442 30 ⟩ ≡

```
fmt.Println("442. kth order propagation")
var nfam, badUP, badFLE, badC, badD, badLemma int
sweep(4, reps, 5, func(c *ctx) {
    nfam++
    for a := 0; a < c.nA; a++ {
        ⟨ Compare level one with unit propagation 31 ⟩
        ⟨ Compare level two with failed literal elimination 32 ⟩
        ⟨ Check the implications of parts (c) and (d) 33 ⟩
    }
    ⟨ Check the lemma that part (e) rests on 34 ⟩
})
⟨ Report on parts (a) through (e) 35 ⟩
⟨ Work out L_k of the clauses R and R′ 36 ⟩
⟨ Run the procedure of part (f) 38 ⟩
```

This code is used in section 4.

31. Unit propagation either reaches the empty clause or it doesn't. In the second case its closure is exactly L_1; in the first case $\vdash_1 \epsilon$ holds, but L_1 need not contain every literal, for a reason explained below.

⟨ Compare level one with unit propagation 31 ⟩ ≡

```
_, bad, trail := c.up(a)
if bad ≠ c.lev[1].contra[a] ∨ (¬bad ∧ trail ≠ c.lev[1].derive[a]) {
    badUP++
}
```

This code is used in section 30.

32. ⟨ Compare level two with failed literal elimination 32 ⟩ ≡

```
dead, forced := c.fle(a)
if dead ≠ c.lev[2].contra[a] ∨ (¬dead ∧ forced ≠ c.lev[2].derive[a]) {
    badFLE++
}
```

This code is used in section 30.

33. Part (c) claims that $F \vdash_k \epsilon$ or $F \vdash_k \bar{l}$ implies $F \mid l \vdash_k \epsilon$, and that $F \vdash_k \epsilon$ implies $F \vdash_{k+1} \epsilon$; part (d) claims that $F \vdash_k l$ implies $F \vdash_{k+1} l$.

⟨ Check the implications of parts (c) and (d) 33 ⟩ ≡

```
for k := 0; k < 4; k++ {
    for l := 2; l < 2 * c.n + 2; l++ {
        if c.free[a] & (1 << l) ≡ 0 {
            continue
        }
        if c.lev[k].contra[a] ∨ c.lev[k].derive[a] & (1 << (l ⊕ 1)) ≠ 0 {
            if ¬c.lev[k].contra[c.assign(a, l)] {
                badC++
            }
        }
    }
    if c.lev[k].contra[a] ∧ ¬c.lev[k + 1].contra[a] {
        badC++
    }
    if c.lev[k].derive[a] ⊘ c.lev[k + 1].derive[a] ≠ 0 {
        badD++
    }
}
```

This code is used in section 30.

34. Part (e) opens with a lemma: if every clause of F is longer than k literals then $L_k(F)$ is empty.

⟨ Check the lemma that part (e) rests on 34 ⟩ ≡

```
for k := 0; k ≤ 4; k++ {
    long := true
    for _, cl := range c.f.cl {
        if bits.OnesCount32(cl) ≤ k {
            long = false
        }
    }
    if long ∧ c.lev[k].derive[0] ≠ 0 {
        badLemma++
    }
}
```

This code is used in section 30.

35. ⟨ Report on parts (a) through (e) 35 ⟩ ≡

```
claim(badUP ≡ 0, "(a) |-_1 is unit propagation: %d families, %d disagreements",
    nfam, badUP)
claim(badFLE ≡ 0, "(b) |-_2 is unit propagation plus failed literals: " +
    "%d disagreements", badFLE)
claim(badC ≡ 0, "(c) the two implications hold: %d exceptions", badC)
claim(badD ≡ 0, "(d) F |-_k l implies F |-_(k+1) l: %d exceptions", badD)
claim(badLemma ≡ 0, "(e) clauses longer than k leave L_k empty: %d exceptions",
    badLemma)
```

This code is used in section 30.

36. The clauses R of 7.2.2.2–(6) are the eight unsatisfiable ternary clauses of 7.1.1–(32); dropping the last one leaves R' of (7), which is satisfied only by covering it with $\{4, \bar{1}, 2\}$. Answer 442(e) says that $L_0(R') = L_1(R') = L_2(R') = \emptyset$ while $L_k(R') = \{\bar{1}, 2, 4\}$ for $k \geq 3$, and it explains why: $R' \vdash_3 \bar{1}$ because $R' \mid 1 \vdash_2 \epsilon$, because $R' \mid 1 \vdash_2 3$ and $R' \mid 1 \vdash_2 \bar{3}$.

⟨ Work out L_k of the clauses R and R' 36 ⟩ ≡

```
rp ≔ newCtx(parseFam(4,"123'␣234'␣341␣41'2␣1'2'3␣2'3'4␣3'4'1'"))
rp.buildLevels(5)
claim(rp.lev[0].derive[0] ≡ 0 ∧ rp.lev[1].derive[0] ≡ 0 ∧
    rp.lev[2].derive[0] ≡ 0,"(e)␣L_0(R')␣=␣L_1(R')␣=␣L_2(R')␣=␣empty")
want ≔ uint32(1) ≪ lit(1,false) | 1 ≪ lit(2,true) | 1 ≪ lit(4,true)
claim(rp.lev[3].derive[0] ≡ want ∧ rp.lev[4].derive[0] ≡ want,
    "(e)␣L_k(R')␣=␣{%s}␣for␣k␣=␣3␣and␣4", litsOf(want,true))
a1 ≔ rp.assign(0,lit(1,true))
claim(rp.lev[2].derive[a1] & (1 ≪ lit(3,true)) ≠ 0 ∧
    rp.lev[2].derive[a1] & (1 ≪ lit(3,false)) ≠ 0 ∧ rp.lev[2].contra[a1],
    "(e)␣R'|1␣|-_2␣3␣and␣R'|1␣|-_2␣3',␣so␣R'|1␣|-_2␣eps")
r ≔ newCtx(parseFam(4,"123'␣234'␣341␣41'2␣1'2'3␣2'3'4␣3'4'1'␣4'12'"))
r.buildLevels(5)
claim(r.lev[2].derive[0] ≡ 0 ∧ r.lev[3].derive[0] ≡ r.free[0],
    "␣␣␣␣␣and␣for␣the␣unsatisfiable␣R:␣L_2␣empty,␣L_3␣everything")
```

This code is used in section 30.

37. The procedure of part 442(f). Part (f) asks for $L_k(F)$ and $F \mid L_k(F)$ in $O(n^{2k-1}m)$ steps. The answer gives a procedure: P_1 is unit propagation; for $k \geq 2$, P_k calls $P_{k-1}(F \mid \bar{l})$ for one literal after another, and as soon as it finds $P_{k-1}(F \mid \bar{l}) = \{\epsilon\}$ it either concludes $\{\epsilon\}$, if $P_{k-1}(F \mid l)$ is $\{\epsilon\}$ too, or restarts on $F \mid l$. Since it commits to l without looking back, the procedure is correct only if the order in which literals are tried doesn't matter, and the answer proves that it doesn't.

⟨ Functions 7 ⟩ +≡

```
func (c *ctx) pk(a, k int, primed bool)(int, uint32, bool) {
    if k ≤ 1 {
        b, bad, got := c.up(a)
        return b, got, bad
    }
    if primed {
        if b, got, dead := c.pk(a, k − 1, primed); dead {
            return b, got, true
        }
    }
    var got uint32
    for progress := true; progress; {
        progress = false
        for l := 2; l < 2 ∗ c.n + 2 ∧ ¬progress; l++ {
            if c.free[a] & (1 ≪ l) ≡ 0 {
                continue
            }
            if _, _, dead := c.pk(c.assign(a, l ⊕ 1), k − 1, primed); ¬dead {
                continue
            }
            if _, _, dead := c.pk(c.assign(a, l), k − 1, primed); dead {
                return a, got | 1 ≪ l, true
            }
            a, got, progress = c.assign(a, l), got | 1 ≪ l, true
        }
    }
    return a, got, false
}
```

38. The procedure agrees with the relation whenever it does not end in a conflict. When it does end in one, the answer says that every literal then belongs to L_k—and at $k \geq 2$ that is what we find, but at $k = 1$ it is not. The reason is the strictly distinct rule. Take $F = \{2, 1\bar{2}, \bar{1}\bar{2}\}$. Unit propagation forces 2, then 1, then $\bar{1}$: so $F \vdash_1 1$, $F \vdash_1 \bar{1}$, $F \vdash_1 2$, and hence $F \vdash_1 \epsilon$. But no sequence of strictly distinct literals ends in $\bar{2}$, because the only prefixes that lead anywhere use the variable 2 at their first step. So $L_1(F) = \{1, \bar{1}, 2\}$, one literal short of everything. The procedure of part (f) treats $k = 1$ separately, by unit propagation, so this costs it nothing; but the parenthesis is worth reading twice.

⟨ Run the procedure of part (f) 38 ⟩ ≡

```
var nord, badOrder, badDead, atOne, atTwo int
var unprimed [4]int
sweep(4, reps, 4, func(c *ctx) {
    for k := 1; k ≤ 3; k++ {
        nord++
        ⟨ Run the procedure both ways and compare 39 ⟩
    }
})
claim(badOrder ≡ 0 ∧ badDead ≡ 0, "(f) P_k finds L_k regardless of order: " +
    "%d runs, %d disagreements", nord, badOrder + badDead)
claim(atTwo ≡ 0, "(f) when P_k ends in a conflict every literal is in L_k: " +
    "%d exceptions at k >= 2, but %d at k = 1", atTwo, atOne)
⟨ Show what the extra call is for 40 ⟩
⟨ Measure the cost of the procedure 42 ⟩
```

This code is used in section 30.

39. ⟨ Run the procedure both ways and compare 39 ⟩ ≡

```
_, got, dead := c.pk(0, k, true)
switch {
case dead ≠ c.lev[k].contra[0]:
    badDead++
case dead:
    if c.lev[k].derive[0] ≠ c.free[0] {
        ⟨ Count where the parenthesis of part (f) fails 41 ⟩
    }
case got ≠ c.lev[k].derive[0]:
    badOrder++
}
if _, _, d := c.pk(0, k, false); d ≠ c.lev[k].contra[0] {
    unprimed[k]++
}
```

This code is used in section 38.

40. Without the extra call the procedure can miss a refutation, and the smallest witness is as small as a witness can be. Let $F = \{\epsilon\}$ on one variable and let $k = 3$. Then P_3 calls $P_2(F \mid x_1)$ and $P_2(F \mid \bar{x}_1)$; each of those has no variable left to try, so each returns its argument rather than $\{\epsilon\}$, and P_3 concludes that all is well. Unit propagation, at the bottom of the recursion, is never reached.

⟨ Show what the extra call is for 40 ⟩ ≡

```
gap := leveled(fam{n: 1, cl: []uint32{0}}, 3)
_, _, raw := gap.pk(0, 3, false)
_, _, fixed := gap.pk(0, 3, true)
claim(¬raw ∧ fixed ∧ gap.lev[3].contra[0],
    "(f) F = {eps} on one variable at k = 3: refuted only with the extra call")
claim(unprimed[1] ≡ 0 ∧ unprimed[2] ≡ 0 ∧ unprimed[3] > 0,
    "(f) without it, P_k misses a refutation %d times at k = 3 and never at " +
      "k = 1 or 2, in the %d runs above", unprimed[3], nord)
```

This code is used in section 38.

41. ⟨ Count where the parenthesis of part (f) fails 41 ⟩ ≡

```
if k ≡ 1 {
    atOne++
} else {
    atTwo++
}
```

This code is used in section 39.

42. The bound $O(n^{2k-1}m)$ is the cost of n rounds of $2n$ calls on P_{k-1}, with unit propagation at the bottom. Counting one step per clause inspected, the measured cost divided by $n^{2k-1}m$ should stay put as n grows.

⟨ Measure the cost of the procedure 42 ⟩ ≡

```
fmt.Println("␣␣␣␣␣␣the␣cost␣of␣P_k,␣in␣clauses␣inspected:")
for k := 2; k ≤ 3; k++ {
    line := fmt.Sprintf("␣␣␣␣␣␣␣␣k=%d:", k)
    for n := 4; n ≤ 8; n++ {
        rng := rand.New(rand.NewPCG(seed, uint64(n)))
        cands := allClauses(n)
        var total, m int
        for i := 0; i < 20; i++ {
            f := randFam(rng, n, cands)
            c := newCtx(f)
            m += len(f.cl)
            steps = 0
            c.pk(0, k, true)
            total += steps
        }
        bound := 1
        for i := 0; i < 2 * k − 1; i++ {
            bound *= n
        }
        line += fmt.Sprintf("␣␣n=%d:␣%d␣steps,␣%.3f␣n^%d␣m", n, total/20,
            float64(total)/float64(bound)/float64(m), 2 * k − 1)
    }
    fmt.Println(line)
}
```

This code is used in section 38.

43. Checking exercise 443. Membership in UC_k and PC_k is a statement about every set L of strictly distinct literals, and the tables have a row for each one, so both tests are short loops. In PC_k's test the literal l must be strictly distinct from L; when L assigns every variable there is no such l, and the definition asks nothing.

⟨ Functions 7 ⟩ +≡

```
func (c *ctx) inUC(k int) bool {
    for a := 0; a < c.nA; a++ {
        if c.unsat[a] ∧ ¬c.lev[k].contra[a] {
            return false
        }
    }
    return true
}
func (c *ctx) inPC(k int) bool {
    for a := 0; a < c.nA; a++ {
        for l := 2; l < 2 * c.n + 2; l++ {
            if c.free[a] & (1 ≪ l) ≡ 0 {
                continue
            }
            if c.implies(a, l) ∧ c.lev[k].derive[a] & (1 ≪ l) ≡ 0 {
                return false
            }
        }
    }
    return true
}
```

44. ⟨ Functions 7 ⟩ +≡

```
func (c *ctx) place(K int)(int, int) {
    uc, pc := −1, −1
    for k := 0; k ≤ K; k++ {
        if uc < 0 ∧ c.inUC(k) {
            uc = k
        }
        if pc < 0 ∧ c.inPC(k) {
            pc = k
        }
    }
    return uc, pc
}
```

45. Part (c) characterizes the second smallest class: F is in UC_0 if and only if it contains all of its prime clauses. A clause is an implicate of F when $F \mid \bar{C}$ is unsatisfiable, and it is prime when no clause obtained by dropping one of its literals is an implicate.

⟨ Functions 7 ⟩ +≡

```
func (c *ctx) isImplicate(m uint32) bool {
    a := 0
    for l := 2; l < 2 * c.n + 2; l++ {
        if m & (1 << l) ≠ 0 {
            if a = c.assign(a, l ⊕ 1); a < 0 {
                return false
            }
        }
    }
    return c.unsat[a]
}
func (c *ctx) containsAllPrimes(cands []uint32) bool {
    for _, m := range cands {
        if ¬c.isImplicate(m) ∨ ¬c.prime(m) {
            continue
        }
        found := false
        for _, cl := range c.f.cl {
            found = found ∨ cl ≡ m
        }
        if ¬found {
            return false
        }
    }
    return true
}
func (c *ctx) prime(m uint32) bool {
    for l := 2; l < 2 * c.n + 2; l++ {
        if m & (1 << l) ≠ 0 ∧ c.isImplicate(m ⊘ (1 << l)) {
            return false
        }
    }
    return true
}
```

46. Part (a)'s witnesses come in pairs. The 2^k clauses that run through every sign pattern of the variables $1, \ldots, k$, with the variable $k+1$ appended to all of them, land in $UC_k \setminus PC_k$; appending $k+1$ to the all-negative one only gives a family in $PC_k \setminus UC_{k-1}$. The answer lists them up to $k = 3$; the program builds them for any k, and checks through $k = 4$.

⟨ Functions 7 ⟩ +≡

```
func signCombos(k int) []uint32 {
    out := []uint32{0}
    for v := 1; v ≤ k; v++ {
        next := make([]uint32, 0, 2 * len(out))
        for _, c := range out {
            next = append(next, c | 1 ≪ lit(v, true), c | 1 ≪ lit(v, false))
        }
        out = next
    }
    return out
}
func witness(k int, uc bool) fam {
    cl, allNeg := signCombos(k), uint32(0)
    for v := 1; v ≤ k; v++ {
        allNeg |= 1 ≪ lit(v, false)
    }
    for i := range cl {
        if uc ∨ cl[i] ≡ allNeg {
            cl[i] |= 1 ≪ lit(k + 1, true)
        }
    }
    return fam{n: k + 1, cl: cl}
}
```

47. ⟨ Check exercise 443 47 ⟩ ≡

```
fmt.Println("443. a hierarchy of hardness")
⟨ Check the witnesses for strictness 48 ⟩
⟨ Check parts (b), (c), and (d) on every small family 49 ⟩
⟨ Check part (e), and find the smallest counterexamples 50 ⟩
⟨ Place the clauses R' in the hierarchy 51 ⟩
```

This code is used in section 4.

48. ⟨ Check the witnesses for strictness 48 ⟩ ≡

```
e := newCtx(fam{n: 1})
e.buildLevels(1)
uc, pc := e.place(1)
ok := uc ≡ 0 ∧ pc ≡ 0
for k := 1; k ≤ 4 ∧ ok; k++ {
    p := newCtx(witness(k, false))
    p.buildLevels(k + 2)
    uc, pc = p.place(k + 2)
    ok = ok ∧ uc ≡ k ∧ pc ≡ k
    u := newCtx(witness(k, true))
    u.buildLevels(k + 2)
    uc, pc = u.place(k + 2)
    ok = ok ∧ uc ≡ k ∧ pc ≡ k + 1
}
claim(ok, "(a) the witnesses sit in PC_k \\ UC_(k-1) and UC_k \\ PC_k, k <= 4")
```

This code is used in section 47.

49. Part (b) says $F \in PC_0$ exactly when F is empty or contains ϵ, and part (d) says that n variables always put F in PC_n. Together with part (c) these are checked on every family of at most five clauses on three variables—a hundred thousand of them—and on random families besides.

⟨ Check parts (b), (c), and (d) on every small family 49 ⟩ ≡

```
var nsmall, badB, badC443, badD443 int
everyFamily(3, 5, 4, func(c *ctx) {
    nsmall++
    empty := len(c.f.cl) ≡ 0
    for _, cl := range c.f.cl {
        empty = empty ∨ cl ≡ 0
    }
    if c.inPC(0) ≠ empty {
        badB++
    }
    if c.inUC(0) ≠ c.containsAllPrimes(allClauses(3)) {
        badC443++
    }
    if ¬c.inPC(3) {
        badD443++
    }
})
claim(badB ≡ 0, "(b) PC_0 is exactly the empty family and those with eps: " +
    "%d families, %d exceptions", nsmall, badB)
claim(badC443 ≡ 0, "(c) UC_0 is exactly the families holding all their prime " +
    "clauses: %d exceptions", badC443)
claim(badD443 ≡ 0, "(d) n variables put F in PC_n: %d exceptions", badD443)
```

This code is used in section 47.

50. Part (e) asks whether n variables put F in UC_{n-1}, and the answer is no. The smallest counterexamples are worth exhibiting: one variable and the two unit clauses, or two variables and all four binary clauses. Both are unsatisfiable without a single unit propagation.

⟨ Check part (e), and find the smallest counterexamples 50 ⟩ ≡

```
one := newCtx(parseFam(1, "1␣1'"))
one.buildLevels(1)
two := newCtx(parseFam(2, "12␣12'␣1'2␣1'2'"))
two.buildLevels(2)
claim(¬one.inUC(0) ∧ ¬two.inUC(1),
    "(e)␣false:␣{1,1'}␣is␣not␣in␣UC_0␣and␣{12,12',1'2,1'2'}␣is␣not␣in␣UC_1")
```

This code is used in section 47.

51. Part (f) places R' exactly: it is in UC_2 but not in PC_2. The reason is the one computed above—$R' \vdash \bar{1}$ semantically, but $L_2(R')$ is empty, so second order propagation cannot produce $\bar{1}$.

⟨ Place the clauses R' in the hierarchy 51 ⟩ ≡

```
rf := newCtx(parseFam(4, "123'␣234'␣341␣41'2␣1'2'3␣2'3'4␣3'4'1'"))
rf.buildLevels(4)
uc2, pc2 := rf.place(4)
a1f := rf.assign(0, lit(1, true))
claim(uc2 ≡ 2 ∧ pc2 ≡ 3, "(f)␣R'␣is␣in␣UC_2␣but␣not␣PC_2")
claim(rf.lev[2].derive[a1f] & (1 ≪ lit(2, true)) ≠ 0 ∧
    rf.lev[2].derive[a1f] & (1 ≪ lit(2, false)) ≠ 0,
    "(f)␣and␣R'|1␣|-_2␣2␣and␣R'|1␣|-_2␣2',␣as␣the␣answer␣says")
```

This code is used in section 47.

52. Checking exercise 444. SLUR propagates, then picks a literal, then looks ahead: if $F \mid l$ does not propagate to a contradiction it commits to l without ever backtracking, else it tries $\bar{l}$, and if both fail it gives up and says 'maybe'. Step E2 says "set l to any literal within F," which is a choice, so the program explores every choice and collects the outcomes that are possible. Since the state of the algorithm is always F restricted by some partial assignment, the exploration is a memoized walk over assignments.

⟨ Functions 7 ⟩ +≡

```
func (c *ctx) slur(wide bool) int {
    a, bad, _ := c.up(0)
    if bad {
        return outUNSAT
    }
    return c.slurE2(a, wide, map[int]int{})
}
func (c *ctx) occurring(a int) uint32 {
    var got uint32
    for _, cl := range c.f.cl {
        if cl & c.tmask[a] ≡ 0 {
            got |= cl ⊘ c.fmask[a]
        }
    }
    return got
}
```

53. ⟨ Global variables 6 ⟩ +≡

```
const (
    outSAT = 1 ≪ iota
    outUNSAT
    outMAYBE
)
```

54. And here is the second place where reading carefully pays. Under the narrow reading, l ranges over the literals that occur in F; under the wide reading it ranges over both literals of every variable that occurs in F. The two differ only when some variable is pure, and the choice decides which branch step E3 looks at first—so the wide reading is the one that makes E3's preference for l over $\bar{l}$ a real choice rather than an accident of which polarity happens to appear.

⟨ Functions 7 ⟩ +≡

```
func (c *ctx) slurE2(a int, wide bool, memo map[int]int) int {
    if c.nres[a] ≡ 0 {
        return outSAT
    }
    if v, ok := memo[a]; ok {
        return v
    }
    out, occ := 0, c.occurring(a)
    for l := 2; l < 2 * c.n + 2; l++ {
        if occ & (1 ≪ l) ≡ 0 ∧ ¬(wide ∧ occ & (1 ≪ (l ⊕ 1)) ≠ 0) {
            continue
        }
        ⟨ Look ahead from l, and commit or give up 55 ⟩
    }
    memo[a] = out
    return out
}
```

55. ⟨ Look ahead from l, and commit or give up 55 ⟩ ≡

```
if b, bad, _ := c.up(c.assign(a, l)); ¬bad {
    out |= c.slurE2(b, wide, memo)
} else if b, bad, _ := c.up(c.assign(a, l ⊕ 1)); ¬bad {
    out |= c.slurE2(b, wide, memo)
} else {
    out |= outMAYBE
}
```

This code is used in section 54.

56. Part (a) is about Horn clauses, possibly renamed: complement some subset of the variables and see whether every clause is left with at most one positive literal. For the small families here, trying all 2^n renamings is the simplest honest test.

⟨ Functions 7 ⟩ +≡

```
func (c *ctx) renamableHorn() bool {
    for s := 0; s < 1 ≪ c.n; s++ {
        ok := true
        for _, cl := range c.f.cl {
            pos := 0
            for v := 1; v ≤ c.n; v++ {
                flip := s & (1 ≪ (v − 1)) ≠ 0
                if cl & (1 ≪ lit(v, ¬flip)) ≠ 0 {
                    pos++
                }
            }
            ok = ok ∧ pos ≤ 1
        }
        if ok {
            return true
        }
    }
    return false
}
```

57. ⟨ Check exercise 444 57 ⟩ ≡

```
fmt.Println("444. the SLUR algorithm")
⟨ Check the four clauses of part (b) 58 ⟩
⟨ Check parts (a) and (c) on every small family 59 ⟩
⟨ Check the interleaving of part (d) 67 ⟩
```

This code is used in section 4.

58. Part (b) wants four clauses on three variables that SLUR always calls 'sat' although they are not renamable Horn clauses; the answer offers $\{12, \bar{2}3, 1\bar{2}\bar{3}, \bar{1}23\}$.

⟨ Check the four clauses of part (b) 58 ⟩ ≡

```
b := newCtx(parseFam(3, "12 2'3 12'3' 1'23"))
b.buildLevels(2)
claim(b.slur(false) ≡ outSAT ∧ b.slur(true) ≡ outSAT ∧ ¬b.renamableHorn(),
    "(b) %v: SLUR always says sat, and it is not renamable Horn", b.f)
```

This code is used in section 57.

59. Part (a) says SLUR never says 'maybe' on renamable Horn clauses, and part (c) says it never says 'maybe' exactly when $F \in UC_1$. The first holds under either reading; the second does not.

⟨ Check parts (a) and (c) on every small family 59 ⟩ ≡

```
var nf444, badHorn, badNarrow, badNarrowSat, badWide, badBack int
var small fam
look := func(c *ctx) {
    nf444++
    uc1, narrow, wide := c.inUC(1), c.slur(false), c.slur(true)
    if c.renamableHorn() ∧ (narrow & outMAYBE ≠ 0 ∨ wide & outMAYBE ≠ 0) {
        badHorn++
    }
    ⟨ Compare the two readings with membership in UC_1 61 ⟩
}
everyFamily(3, 5, 2, look)
sweep(5, reps, 2, look)
claim(badHorn ≡ 0, "(a) renamable Horn clauses never get a maybe: " +
    "%d families, %d exceptions", nf444, badHorn)
claim(badWide ≡ 0 ∧ badBack ≡ 0,
    "(c) with l ranging over both literals of F's variables: %d exceptions",
    badWide + badBack)
claim(badNarrow > 0, "(c) with l ranging only over the literals that occur " +
    "in F, the claim fails: %d families decided without being in UC_1, " +
    "all %d of them satisfiable", badNarrow, badNarrowSat)
fmt.Printf("         the smallest is %v, undecided at L = %s\n",
    small, smallWitness)
```

This code is used in section 57.

60. ⟨ Global variables 6 ⟩ +≡

```
var smallWitness string
```

61. ⟨ Compare the two readings with membership in UC_1 61 ⟩ ≡

```
if wide & outMAYBE ≡ 0 ∧ ¬uc1 {
    badWide++
}
if wide & outMAYBE ≠ 0 ∧ uc1 {
    badBack++
}
if narrow & outMAYBE ≡ 0 ∧ ¬uc1 {
    badNarrow++
    if ¬c.unsat[0] {
        badNarrowSat++
    }
    if small.cl ≡ Λ ∨ len(c.f.cl) < len(small.cl) {
        small = c.f
        ⟨ Record the assignment that makes UC_1 fail 62 ⟩
    }
}
```

This code is used in section 59.

62. ⟨ Record the assignment that makes UC_1 fail 62 ⟩ ≡

```
smallWitness = "?"
for a := 0; a < c.nA; a++ {
    if ¬c.unsat[a] ∨ c.lev[1].contra[a] {
        continue
    }
    var w strings.Builder
    for v := 1; v ≤ c.n; v++ {
        if d := c.digit(a, v); d ≠ 0 {
            w.WriteString(litName(lit(v, d ≡ 1)))
        }
    }
    smallWitness = w.String( )
    break
}
```

This code is used in section 61.

63. Part (d) asks for a linear time implementation, and suggests interleaving the propagation on $F \mid l$ with the propagation on $F \mid \bar{l}$, stopping as soon as one of them finishes without reaching the empty clause. The work is then at most twice the work of the shorter branch. Committing to whichever branch finished first can differ from E3 as written, which always prefers l; but under the wide reading of E2 both polarities are offered anyway, so the set of outcomes cannot change—and that is a claim one can test.

⟨ Functions 7 ⟩ +≡

```
func (c *ctx) interleaved(a int, wide bool, memo map[int]int) int {
    if c.nres[a] ≡ 0 {
        return outSAT
    }
    if v, ok := memo[a]; ok {
        return v
    }
    out, occ := 0, c.occurring(a)
    for l := 2; l < 2 * c.n + 2; l++ {
        if occ & (1 ≪ l) ≡ 0 ∧ ¬(wide ∧ occ & (1 ≪ (l ⊕ 1)) ≠ 0) {
            continue
        }
        ⟨ Race the two branches against each other 64 ⟩
    }
    memo[a] = out
    return out
}
```

64. One propagation step in one branch, then one in the other, until a branch runs out of units. *race* returns the winning state, or -1 if both branches died.

⟨ Race the two branches against each other 64 ⟩ ≡

```
if w := c.race(a, l); w ≥ 0 {
    out |= c.interleaved(w, wide, memo)
} else {
    out |= outMAYBE
}
```

This code is used in section 63.

65. ⟨ Functions 7 ⟩ +≡

```
func (c *ctx) race(a, l int) int {
    side, dead := [2]int{c.assign(a, l), c.assign(a, l ⊕ 1)}, [2]bool{ }
    for {
        for i := 0; i < 2; i++ {
            if dead[i] {
                continue
            }
            ⟨ Take one propagation step on side i 66 ⟩
        }
        if dead[0] ∧ dead[1] {
            return −1
        }
    }
}
```

66. ⟨ Take one propagation step on side i 66 ⟩ ≡

```
if c.eps[side[i]] {
    dead[i] = true
    continue
}
found := −1
for _, cl := range c.f.cl {
    steps++
    if cl & c.tmask[side[i]] ≠ 0 {
        continue
    }
    if r := cl ⊘ c.fmask[side[i]]; bits.OnesCount32(r) ≡ 1 {
        found = bits.TrailingZeros32(r)
        break
    }
}
if found < 0 {
    return side[i]
}
side[i], forced = c.assign(side[i], found), forced + 1
```

This code is used in section 65.

67. ⟨ Check the interleaving of part (d) 67 ⟩ ≡

```
var nd, badRace, naive, raced int
sweep(4, reps, 2, func(c *ctx) {
    nd++
    steps = 0
    plain := c.slur(true)
    naive += steps
    steps = 0
    a, bad, _ := c.up(0)
    fast := outUNSAT
    if ¬bad {
        fast = c.interleaved(a, true, map[int]int{})
    }
    raced += steps
    if plain ≠ fast {
        badRace++
    }
})
claim(badRace ≡ 0, "(d) interleaving the two lookaheads decides the same " +
    "families: %d of them, %d differences", nd, badRace)
fmt.Printf("        clauses inspected: %d as written, %d interleaved\n",
    naive, raced)
```

This code is also defined in section 71.

This code is used in section 57.

68. On random families the interleaving saves nothing—it costs a little, since it starts a branch it may not need. Its point is the worst case, and a family that shows it is this: m variables $y_1, \ldots, y_m$, each of which implies the head of one implication chain $x_1 \to x_2 \to \cdots \to x_m$, and a contradiction at the far end of the chain. Setting y_j true forces the whole chain before anything goes wrong; setting it false forces nothing at all. SLUR fixes one y_j per round, so a lookahead that runs one branch to the end before starting the other does $\Theta(m^2)$ work on a family of $O(m)$ cells, while the interleaved lookahead notices after a step or two that the other branch is already complete.

⟨ Functions 7 ⟩ +≡

```
func chain(m int) fam {
    f := fam{n: 2 * m + 1}
    for j := 1; j ≤ m; j++ {
        f.cl = append(f.cl, 1 ≪ lit(j, false) | 1 ≪ lit(m + 1, true))
    }
    for i := 1; i < m; i++ {
        f.cl = append(f.cl, 1 ≪ lit(m + i, false) | 1 ≪ lit(m + i + 1, true))
    }
    f.cl = append(f.cl, 1 ≪ lit(2 * m, false) | 1 ≪ lit(2 * m + 1, true),
        1 ≪ lit(2 * m, false) | 1 ≪ lit(2 * m + 1, false))
    return f
}
```

69. To count the work we need a single run rather than the whole tree of runs, so *oneRun* always takes the lowest-numbered variable that is still there, and looks ahead from its positive literal. The work of a lookahead is the number of literals it forces; with watch lists that is what it costs.

```
⟨ Functions 7 ⟩ +≡
  func (c *ctx) oneRun(interleave bool) int {
      forced = 0
      a, bad, _ := c.up(0)
      for ¬bad ∧ c.nres[a] ≠ 0 {
          occ, v := c.occurring(a), 1
          for occ & (3 ≪ lit(v, true)) ≡ 0 {
              v++
          }
          l := lit(v, true)
          if interleave {
              if a = c.race(a, l); a < 0 {
                  break
              }
              continue
          }
          ⟨ Take the two lookaheads one after the other 70 ⟩
      }
      return forced
  }
```

70.

```
⟨ Take the two lookaheads one after the other 70 ⟩ ≡
  if b, dead, _ := c.up(c.assign(a, l)); ¬dead {
      a = b
  } else if b, dead, _ := c.up(c.assign(a, l ⊕ 1)); ¬dead {
      a = b
  } else {
      break
  }
```

This code is used in section 69.

71.

```
⟨ Check the interleaving of part (d) 67 ⟩ +≡
  fmt.Println("␣␣␣␣␣␣␣␣on␣the␣chain␣family,␣literals␣forced␣in␣lookaheads:")
  var lastPlain, lastFast int
  for m := 1; m ≤ 4; m++ {
      c := newCtx(chain(m))
      cells := 0
      for _, cl := range c.f.cl {
          cells += bits.OnesCount32(cl)
      }
      lastPlain, lastFast = c.oneRun(false), c.oneRun(true)
      fmt.Printf("␣␣␣␣␣␣␣␣␣␣m=%d:␣%d␣variables,␣%d␣cells;␣%d␣as␣written,␣" +
          "%d␣interleaved\n", m, c.n, cells, lastPlain, lastFast)
  }
  claim(lastPlain ≥ 2 * lastFast, "(d)␣at␣m␣=␣4␣the␣interleaved␣lookaheads␣force␣" +
      "%d␣literals␣against␣%d", lastFast, lastPlain)
```

72. The honesty test of page 289. The text just before the section on symmetry breaking asks a representation F of a constraint f to be *honest*: whenever L is a set of n literals that fully characterizes a solution of $f(x_1,\ldots,x_n)=1$, the clauses $F \mid L$ must be easy to satisfy, "using the SLUR algorithm of exercise 444." And it adds that the test "is automatically passed whenever every clause of F contains at most one negated auxiliary variable."

That is a claim about SLUR, so it belongs here. The program builds random families over two primary variables and three auxiliary ones, with at most one negated auxiliary per clause, and asks SLUR about $F \mid L$ for every assignment L of the primary variables.

⟨ Check the honesty test of page 289 72 ⟩ ≡

```
fmt.Println("289. honest representations")
var nh, badHonest int
rng, cands := rand.New(rand.NewPCG(seed, 289)), allClauses(6)
for i := 0; i < reps; i++ {
    ⟨ Build a family with at most one negated auxiliary variable 73 ⟩
    nh++
    if leveled(f, 1).stuck(2) > 0 {
        badHonest++
    }
}
claim(badHonest ≡ 0, "at most one negated auxiliary variable implies honesty: " +
    "%d families, %d of them leave SLUR stuck", nh, badHonest)
```

This code is also defined in section 76.

This code is used in section 4.

73. Variables 1 and 2 are the primary ones, variables 3 through 6 are auxiliary. Both samples are drawn from the same urn, all 3^6 clauses; one of them then throws away the clauses with two or more negated auxiliary variables.

⟨ Build a family with at most one negated auxiliary variable 73 ⟩ ≡

```
f := fam{n: 6}
for j, seen := 0, map[uint32]bool{}; j < 10; j++ {
    cl, neg := cands[rng.IntN(len(cands))], 0
    for v := 3; v ≤ 6; v++ {
        if cl & (1 ≪ lit(v, false)) ≠ 0 {
            neg++
        }
    }
    if cl ≠ 0 ∧ neg ≤ 1 ∧ ¬seen[cl] {
        seen[cl] = true
        f.cl = append(f.cl, cl)
    }
}
```

This code is used in section 72.

74. A set L of literals that fully characterizes a solution is an assignment of the primary variables that leaves the clauses satisfiable, and the question is whether SLUR can then finish the job without backtracking.

⟨ Functions 7 ⟩ +≡

```
func (c *ctx) stuck(p int) int {
    n := 0
    for t := 0; t < 1 ≪ p; t++ {
        a := 0
        for v := 1; v ≤ p; v++ {
            a = c.assign(a, lit(v, t & (1 ≪ (v − 1)) ≠ 0))
        }
        if c.unsat[a] {
            continue
        }
        if leveled(c.restrict(a), 1).slur(true) & outMAYBE ≠ 0 {
            n++
        }
    }
    return n
}
```

75. To hand $F \mid L$ to SLUR as a family in its own right, its clauses have to be shortened; the variables keep their numbers, which costs a little room and saves a lot of confusion.

⟨ Functions 7 ⟩ +≡

```
func (c *ctx) restrict(a int) fam {
    g := fam{n: c.n}
    for _, cl := range c.f.cl {
        if cl & c.tmask[a] ≡ 0 {
            g.cl = append(g.cl, cl ⊘ c.fmask[a])
        }
    }
    return g
}
```

76. Random families are useless as a control: lifting the restriction, they still practically never leave SLUR stuck, so the condition has to be shown to bite by hand. Let $H = \{34, \bar{3}4, 3\bar{4}, 2\bar{3}\bar{4}\}$ on the auxiliary variables 2, 3, 4—the smallest family on which SLUR can get stuck—and let the primary variable 1 guard it: $F = \{\bar{1} \vee C \mid C \in H\}$. Both values of x_1 leave F satisfiable, so $L = \{1\}$ characterizes a solution; and $F \mid L$ is H itself, where SLUR commits to $\bar{2}$ and is lost. The offending clause $1\bar{2}\bar{3}\bar{4}$ has two negated auxiliary variables, which is just what page 289 rules out.

⟨ Check the honesty test of page 289 72 ⟩ +≡

```
witness := leveled(parseFam(4, "1'34␣1'3'4␣1'34'␣1'23'4'"), 1)
claim(witness.stuck(1) > 0, "but␣two␣of␣them␣in␣one␣clause␣can:␣%v␣leaves␣" +
    "SLUR␣stuck␣at␣L␣=␣{1}", witness.f)
```

77. The sat package as an oracle. Everything above rests on *unsat*, which a loop over the 2^n total assignments fills in. The package can answer the same question, with the partial assignment handed over as assumption literals, and it answers it the way a real solver would—by propagating, learning, and keeping what it learned for the next call. If the two ever disagree, one of them is wrong.

Three things are checked at once: the verdict; that a reported model really satisfies the clauses and respects the assumptions; and that the assumptions *Failed* blames are enough by themselves to make the clauses unsatisfiable.

```
⟨ Check the tables against the sat package 77 ⟩ ≡
    fmt.Println("the␣sat␣package,␣asked␣the␣same␣questions")
    var ncall, badVerdict, badModel, badFailed int
    sweep(5, reps/3, 1, func(c *ctx) {
        s := c.solver()
        for a := 0; a < c.nA; a++ {
            ⟨ Ask the package about one partial assignment 79 ⟩
        }
    })
    claim(badVerdict ≡ 0, "%d␣calls␣with␣assumptions,␣%d␣disagreements␣about␣" +
        "satisfiability", ncall, badVerdict)
    claim(badModel ≡ 0, "every␣model␣satisfies␣the␣clauses␣and␣the␣assumptions:␣" +
        "%d␣exceptions", badModel)
    claim(badFailed ≡ 0, "every␣failed␣set␣is␣by␣itself␣unsatisfiable:␣" +
        "%d␣exceptions", badFailed)
```

This code is used in section 4.

78.

```
⟨ Functions 7 ⟩ +≡
    func (c *ctx) solver() *sat.Solver {
        s := sat.New()
        c.lits = make([]sat.Lit, c.n + 1)
        for v := 1; v ≤ c.n; v++ {
            c.lits[v] = s.Lookup(strconv.Itoa(v))
        }
        for _, cl := range c.f.cl {
            var ls []sat.Lit
            for l := 2; l < 2 * c.n + 2; l++ {
                if cl & (1 ≪ l) ≠ 0 {
                    ls = append(ls, c.lit(l))
                }
            }
            s.AddClause(ls...)
        }
        return s
    }
    func (c *ctx) lit(l int) sat.Lit {
        if l & 1 ≡ 1 {
            return c.lits[l ≫ 1].Not()
        }
        return c.lits[l ≫ 1]
    }
```

79. ⟨ Ask the package about one partial assignment 79 ⟩ ≡

```
var asm []sat.Lit
for v := 1; v ≤ c.n; v++ {
    if d := c.digit(a, v); d ≠ 0 {
        asm = append(asm, c.lit(lit(v, d ≡ 1)))
    }
}
st, err := s.Solve(context.Background(), asm...)
ncall++
if err ≠ Λ ∨ (st ≡ sat.Unsat) ≠ c.unsat[a] {
    badVerdict++
    continue
}
if st ≡ sat.Sat {
    ⟨ Check the model 80 ⟩
} else {
    ⟨ Check the failed assumptions 81 ⟩
}
```

This code is used in section 77.

80. ⟨ Check the model 80 ⟩ ≡

```
var tm uint32
for v := 1; v ≤ c.n; v++ {
    tm |= 1 ≪ lit(v, s.Value(c.lits[v]))
}
if tm & c.fmask[a] ≠ 0 {
    badModel++
}
for _, cl := range c.f.cl {
    if cl & tm ≡ 0 {
        badModel++
        break
    }
}
```

This code is used in section 79.

81. The package returns the assumptions that were responsible; an empty answer means the clauses are unsatisfiable on their own. Either way, restricting by just those literals must leave an unsatisfiable family.

⟨ Check the failed assumptions 81 ⟩ ≡

```
b := 0
for _, l := range s.Failed() {
    v := l.Var()
    if v < 1 ∨ v > c.n {
        b = −1
        break
    }
    if b = c.assign(b, lit(v, ¬l.IsNeg())); b < 0 {
        break
    }
}
if b < 0 ∨ ¬c.unsat[b] {
    badFailed++
}
```

This code is used in section 79.

82. Random and exhaustive families. Two ways of producing families to test: a random sample for each number of variables, and every family of at most a few clauses. Both hand the caller a context with its levels already built, since building them is the expensive part and every caller wants them.

⟨ Functions 7 ⟩ +≡

```
func leveled(f fam, levels int) *ctx {
    c := newCtx(f)
    c.buildLevels(levels)
    return c
}
func randFam(rng *rand.Rand, n int, cands []uint32) fam {
    f, seen := fam{n: n}, map[uint32]bool{}
    for j, m := 0, 1 + rng.IntN(9); j < m; j++ {
        if cl := cands[rng.IntN(len(cands))]; ¬seen[cl] {
            seen[cl] = true
            f.cl = append(f.cl, cl)
        }
    }
    return f
}
```

83. ⟨ Functions 7 ⟩ +≡

```
func sweep(maxN, count, levels int, do func(*ctx)) {
    for n := 1; n ≤ maxN; n++ {
        cands := allClauses(n)
        rng := rand.New(rand.NewPCG(seed, uint64(n)))
        for i := 0; i < count; i++ {
            do(leveled(randFam(rng, n, cands), levels))
        }
    }
}
func everyFamily(n, max, levels int, do func(*ctx)) {
    cands := allClauses(n)
    var rec func(start int, cl []uint32)
    rec = func(start int, cl []uint32) {
        if len(cl) > 0 {
            do(leveled(fam{n: n, cl: cl}, levels))
        }
        if len(cl) ≡ max {
            return
        }
        for i := start; i < len(cands); i++ {
            rec(i + 1, append(append([]uint32{}, cl...), cands[i]))
        }
    }
    rec(0, Λ)
}
```

84. Index.

a: 15, 18, 20, 21, 23, 24, 25, 26, 27, 28, 30, 31, 32, 33, 37, 43, 45, 52, 54, 55, 62, 63, 64, 65, 67, 69, 70, 74, 75, 77, 79, 80.
AddClause: 78.
allClauses: 13, 42, 49, 72, 83.
allNeg: 46.
append: 12, 13, 25, 46, 68, 73, 75, 78, 79, 82, 83.
args: 7.
asm: 79.
assign: 21, 25, 27, 28, 33, 36, 37, 45, 51, 55, 65, 66, 70, 74, 81.
atOne: 38, 41.
atTwo: 38, 41.
a1: 36.
a1f: 51.
b: 11, 20, 21, 25, 37, 55, 58, 70, 81.
Background: 79.
bad: 28, 31, 37, 52, 55, 67, 69.
badB: 49.
badBack: 59, 61.
badC: 30, 33, 35.
badC443: 49.
badD: 30, 33, 35.
badDead: 38, 39.
badD443: 49.
badFailed: 77, 81.
badFLE: 30, 32, 35.
badHonest: 72.
badHorn: 59.
badLemma: 30, 34, 35.
badModel: 77, 80.
badNarrow: 59, 61.
badNarrowSat: 59, 61.
badOrder: 38, 39.
badRace: 67.
badUP: 30, 31, 35.
badVerdict: 77, 79.
badWide: 59, 61.
bits: 27, 34, 66, 71.
bound: 42.
Builder: 11, 62.
buildLevels: 23, 36, 48, 50, 51, 58, 82.
c: 11, 12, 13, 15, 16, 17, 18, 19, 20, 21, 23, 24, 25, 26, 27, 28, 30, 31, 32, 33, 34, 37, 38, 39, 42, 43, 44, 45, 46, 49, 52, 54, 55, 56, 59, 61, 62, 63, 64, 65, 66, 67, 69, 70, 71, 74, 75, 77, 78, 79, 80, 81, 82.
cands: 42, 45, 72, 73, 82, 83.
cells: 71.
chain: 68, 71.
cl: 9, 11, 12, 17, 18, 27, 34, 40, 42, 45, 46, 49, 52, 56, 61, 66, 68, 71, 73, 75, 78, 80, 82, 83.
claim: 7, 35, 36, 38, 40, 48, 49, 50, 51, 58, 59, 67, 71, 72, 76, 77.
commas: 11.
containsAllPrimes: 45, 49.
context: 79.
contra: 3, 22, 23, 24, 25, 26, 31, 32, 33, 36, 39, 40, 43, 62.
count: 83.
ctx: 14, 15, 21, 23, 24, 27, 28, 30, 37, 38, 43, 44, 45, 49, 52, 54, 56, 59, 63, 65, 67, 69, 74, 75, 77, 78, 82, 83.
d: 20, 39, 62, 79.
dead: 28, 32, 37, 39, 65, 66, 70.
derive: 3, 22, 23, 24, 31, 32, 33, 34, 36, 39, 43, 51.
digit: 15, 18, 20, 21, 25, 62, 79.
do: 83.
e: 48.
empty: 49.
eps: 14, 18, 19, 23, 27, 66.
err: 79.
everyFamily: 49, 59, 83.
Exit: 8.
f: 11, 12, 14, 15, 17, 18, 27, 34, 42, 45, 49, 52, 56, 58, 61, 66, 68, 71, 72, 73, 75, 76, 78, 80, 82.
Failed: 77, 81.
failures: 6, 7, 8.
fam: 9, 11, 12, 14, 15, 40, 46, 48, 59, 68, 73, 75, 82, 83.
fast: 67.
Fields: 12.
fixed: 40.
flag: 5.
fle: 28, 32.
flip: 56.
fm: 18.
fmask: 14, 18, 19, 27, 52, 66, 75, 80.
fmt: 7, 8, 30, 42, 47, 57, 59, 67, 71, 72, 77.
forced: 27, 29, 32, 66, 69.
format: 7.
found: 27, 45, 66.
fr: 18.
free: 14, 18, 19, 23, 28, 33, 36, 37, 39, 43.

g: 28, 75.
gap: 40.
got: 24, 25, 26, 27, 28, 37, 39, 52.
i: 11, 12, 16, 42, 46, 65, 66, 68, 72, 83.
implies: 21, 43.
inPC: 43, 44, 49.
interleave: 69.
interleaved: 63, 64, 67.
IntN: 73, 82.
IntVar: 5.
inUC: 43, 44, 49, 50, 59.
isImplicate: 45.
IsNeg: 81.
Itoa: 10, 78.
j: 68, 73, 82.
Join: 11.
K: 23, 44.
k: 23, 33, 34, 37, 38, 39, 41, 42, 43, 44, 46, 48.
l: 9, 10, 11, 21, 25, 28, 33, 37, 43, 45, 54, 55, 63, 64, 65, 69, 70, 78, 81.
lastFast: 71.
lastPlain: 71.
Len: 11.
len: 11, 12, 13, 25, 42, 46, 49, 61, 73, 82, 83.
lev: 14, 23, 31, 32, 33, 34, 36, 39, 40, 43, 51, 62.
leveled: 40, 72, 74, 76, 82, 83.
levels: 82, 83.
line: 42.
lit: 10, 12, 13, 17, 18, 25, 26, 36, 46, 51, 56, 62, 68, 69, 73, 74, 78, 79, 80, 81.
Lit: 14, 78, 79.
litName: 9, 10, 11, 62.
lits: 14, 78, 80.
litsOf: 11, 36.
long: 34.
look: 59.
Lookup: 78.
ls: 78.
lvl: 14, 22, 23, 24.
l0: 23.
m: 7, 11, 42, 45, 68, 71, 82.
main: 4.
make: 11, 13, 16, 17, 19, 23, 24, 46, 78.
mark: 7.
max: 83.
maxN: 83.
memo: 54, 55, 63, 64.
mode: 5, 6, 7.
n: 9, 12, 13, 14, 15, 16, 17, 18, 20, 25, 26, 28, 33, 37, 40, 42, 43, 45, 46, 48, 54, 56, 62, 63, 68, 71, 73, 74, 75, 78, 79, 80, 81, 82, 83.
nA: 14, 16, 18, 19, 23, 24, 30, 43, 62, 77.
naive: 67.
narrow: 59, 61.
nb: 25.
ncall: 77, 79.
nd: 67.
neg: 73.
New: 42, 72, 78, 83.
newCtx: 15, 36, 42, 48, 50, 51, 58, 71, 82.
NewPCG: 42, 72, 83.
next: 13, 24, 26, 46.
nextLevel: 23, 24.
nfam: 30, 35.
nf444: 59.
nh: 72.
nord: 38, 40.
Not: 78.
now: 14, 25.
nres: 14, 18, 19, 54, 63, 69.
nsmall: 49.
occ: 54, 63, 69.
occurring: 52, 54, 63, 69.
ok: 7, 20, 48, 54, 56, 63.
on: 4, 7.
one: 50.
oneRun: 69, 71.
OnesCount32: 27, 34, 66, 71.
oracle: 3, 17.
os: 8.
out: 13, 46, 54, 55, 63, 64.
outMAYBE: 53, 55, 59, 61, 64, 74.
outSAT: 53, 54, 58, 63.
outUNSAT: 52, 53, 67.
p: 48, 74.
Parse: 5.
parseFam: 11, 12, 36, 50, 51, 58, 76.
parts: 11.
pc: 44, 48.
pc2: 51.
pk: 37, 39, 40, 42.
place: 44, 48, 51.
plain: 67.
pos: 56.
positive: 10, 12.
pow3: 14, 15, 16, 21.

prev: 24, 25, 26.
prime: 45.
primed: 37.
Printf: 7, 8, 59, 67, 71.
Println: 8, 30, 42, 47, 57, 71, 72, 77.
progress: 28, 37.
qi: 25.
queue: 14, 25.
r: 27, 36, 66.
race: 64, 65, 69.
raced: 67.
Rand: 82.
rand: 42, 72, 82, 83.
randFam: 42, 82, 83.
raw: 40.
rec: 83.
renamableHorn: 56, 58, 59.
reps: 5, 6, 30, 38, 59, 67, 72, 77.
restrict: 74, 75.
rf: 51.
rng: 42, 72, 73, 82, 83.
rp: 36.
s: 10, 12, 56, 77, 78, 79, 80, 81.
Sat: 79.
sat: 14, 78, 79.
satT: 17, 20.
seed: 5, 6, 42, 72, 83.
seen: 73, 82.
side: 65, 66.
signCombos: 46.
slur: 52, 58, 59, 67, 74.
slurE2: 52, 54, 55.
small: 59, 61.
smallWitness: 59, 60, 62.
Solve: 79.
solver: 77, 78.
Solver: 78.
Sprintf: 7, 42.
st: 79.
stamp: 14, 19, 25.
start: 83.
steps: 27, 29, 42, 66, 67.
strconv: 10, 78.
String: 11, 62.
strings: 11, 12, 62.
StringVar: 5.
stuck: 72, 74, 76.
sweep: 30, 38, 59, 67, 77, 83.
t: 17, 20, 74.
tm: 17, 18, 80.
tmask: 14, 18, 19, 27, 52, 66, 75.
total: 42.
trail: 31.
TrailingZeros32: 27, 66.
two: 50.
u: 48.
uc: 44, 46, 48.
uc1: 59, 61.
uc2: 51.
Uint64Var: 5.
unprimed: 38, 39, 40.
Unsat: 79.
unsat: 14, 19, 20, 21, 43, 45, 61, 62, 74, 77, 79, 81.
up: 27, 28, 31, 37, 52, 55, 67, 69, 70.
v: 10, 12, 13, 15, 17, 18, 20, 21, 25, 26, 46, 54, 56, 62, 63, 69, 73, 74, 78, 79, 80, 81.
Value: 80.
Var: 81.
w: 12, 62, 64.
want: 36.
wide: 52, 54, 55, 59, 61, 63, 64.
witness: 46, 48, 76.
WriteByte: 11.
WriteString: 11, 62.

⟨ Ask the package about one partial assignment 79 ⟩ Used in section 77
⟨ Build a family with at most one negated auxiliary variable 73 ⟩ Used in section 72
⟨ Check exercise 442 30 ⟩ Used in section 4
⟨ Check exercise 443 47 ⟩ Used in section 4
⟨ Check exercise 444 57 ⟩ Used in section 4
⟨ Check part (e), and find the smallest counterexamples 50 ⟩ Used in section 47
⟨ Check parts (a) and (c) on every small family 59 ⟩ Used in section 57
⟨ Check parts (b), (c), and (d) on every small family 49 ⟩ Used in section 47
⟨ Check the failed assumptions 81 ⟩ Used in section 79
⟨ Check the four clauses of part (b) 58 ⟩ Used in section 57
⟨ Check the honesty test of page 289 72 ⟩ Used in section 4
⟨ Check the implications of parts (c) and (d) 33 ⟩ Used in section 30
⟨ Check the interleaving of part (d) 67 ⟩ Used in section 57
⟨ Check the lemma that part (e) rests on 34 ⟩ Used in section 30
⟨ Check the model 80 ⟩ Used in section 79
⟨ Check the tables against the `sat` package 77 ⟩ Used in section 4
⟨ Check the witnesses for strictness 48 ⟩ Used in section 47
⟨ Collect the literals derivable from a at this level 25 ⟩ Used in section 24
⟨ Compare level one with unit propagation 31 ⟩ Used in section 30
⟨ Compare level two with failed literal elimination 32 ⟩ Used in section 30
⟨ Compare the two readings with membership in UC_1 61 ⟩ Used in section 59
⟨ Count where the parenthesis of part (f) fails 41 ⟩ Used in section 39
⟨ Decide whether $F \mid L$ is satisfiable 20 ⟩ Used in section 18
⟨ Decide whether the empty clause follows 26 ⟩ Used in section 24
⟨ Find the total assignments that satisfy F 17 ⟩ Used in section 15
⟨ Functions 7 ⟩ Used in section 4
⟨ Global variables 6 ⟩ Used in section 4
⟨ Look ahead from l, and commit or give up 55 ⟩ Used in section 54
⟨ Make room for the tables 19 ⟩ Used in section 18
⟨ Measure the cost of the procedure 42 ⟩ Used in section 38
⟨ Place the clauses R' in the hierarchy 51 ⟩ Used in section 47
⟨ Race the two branches against each other 64 ⟩ Used in section 63
⟨ Read the command line 5 ⟩ Used in section 4
⟨ Record the assignment that makes UC_1 fail 62 ⟩ Used in section 61
⟨ Report on parts (a) through (e) 35 ⟩ Used in section 30
⟨ Run the procedure both ways and compare 39 ⟩ Used in section 38
⟨ Run the procedure of part (f) 38 ⟩ Used in section 30
⟨ Say how it went 8 ⟩ Used in section 4
⟨ Set up the powers of three 16 ⟩ Used in section 15
⟨ Show what the extra call is for 40 ⟩ Used in section 38
⟨ Tabulate every partial assignment 18 ⟩ Used in section 15
⟨ Take one propagation step on side i 66 ⟩ Used in section 65
⟨ Take the two lookaheads one after the other 70 ⟩ Used in section 69
⟨ Types 9 ⟩ Used in section 4
⟨ Work out L_k of the clauses R and R' 36 ⟩ Used in section 30

The UC and PC hierarchy

	Section	Page
Introduction	1	1
Clauses, families, and partial assignments	9	4
The relation of exercise 442	22	10
Unit propagation, and failing literals	27	12
Checking exercise 442	30	14
The procedure of part 442(f)	37	17
Checking exercise 443	43	21
Checking exercise 444	52	26
The honesty test of page 289	72	34
The `sat` package as an oracle	77	36
Random and exhaustive families	82	39
Index	84	40